RAMAIAH
Institute of Technology

M.S. Ramaiah Institute of Technology
(Autonomous Institute, Affiliated to VTU)
Department of Artificial Intelligence and Data Science

Course Name: Introduction to Artificial Intelligence(AI)
Course Code: AI35/AD35
Credits: 3:0:0

UNIT -4

RAMAIAH
Institute of Technology

Syllabus

Automated Planning –

- Machine Learning: Learning from examples
- Forms of Learning
- Reinforcement Learning: Learning from Rewards
- Passive Reinforcement Learning
- Active Reinforcement Learning
- Generalization in Reinforcement Learning
- Policy Search, Applications of Reinforcement Learning
- Case Study: Reinforcement Learning Models and Algorithms for Diabetes Management

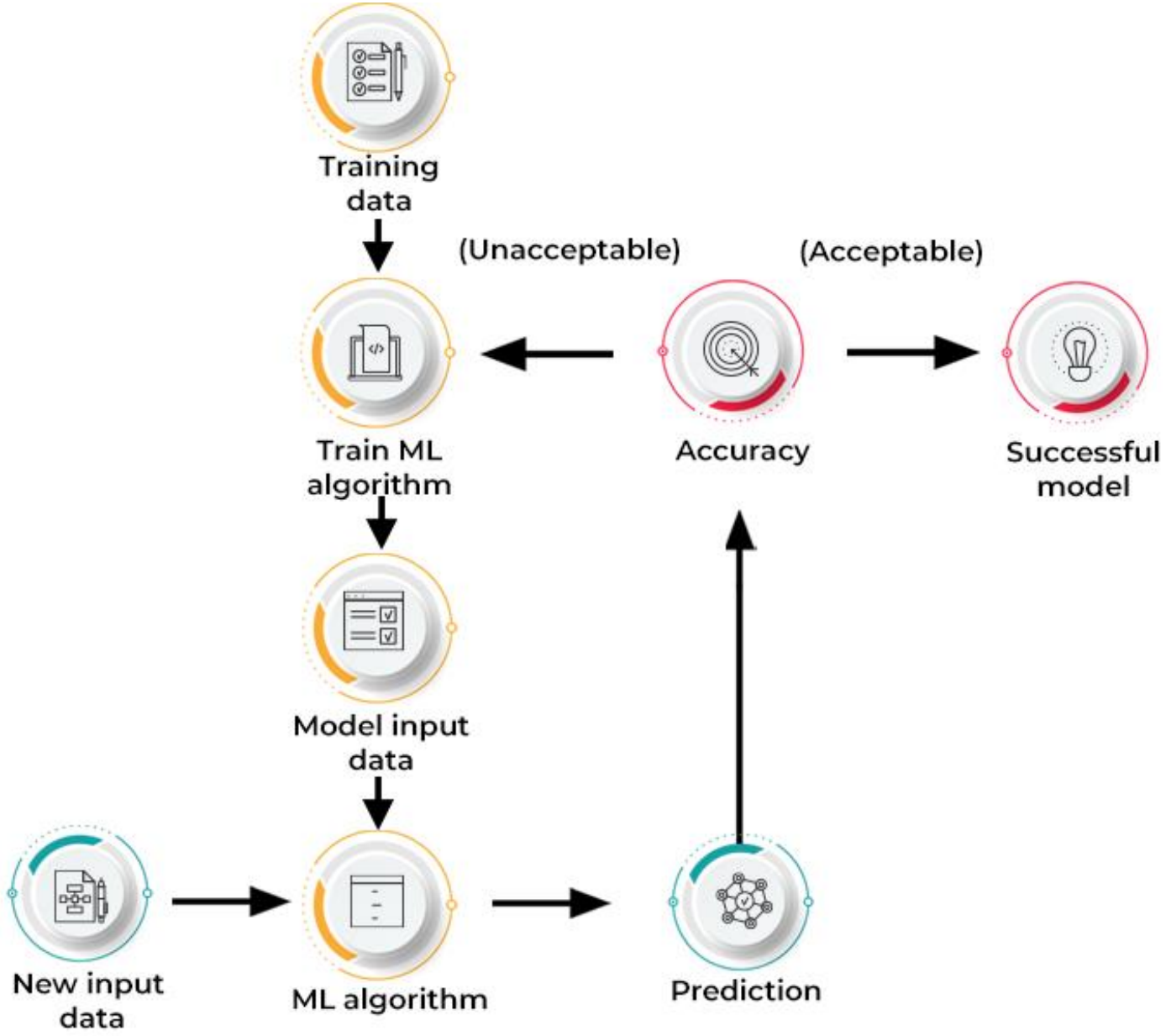
Introduction to Machine Learning

- Machine learning is a **subfield of artificial intelligence**, which is broadly defined as the capability of a machine to imitate intelligent human behavior.
- Machine learning (ML) is defined as a discipline of artificial intelligence (AI) that provides machines the ability to **automatically learn from data and past experiences to identify patterns and make predictions with minimal human intervention.**

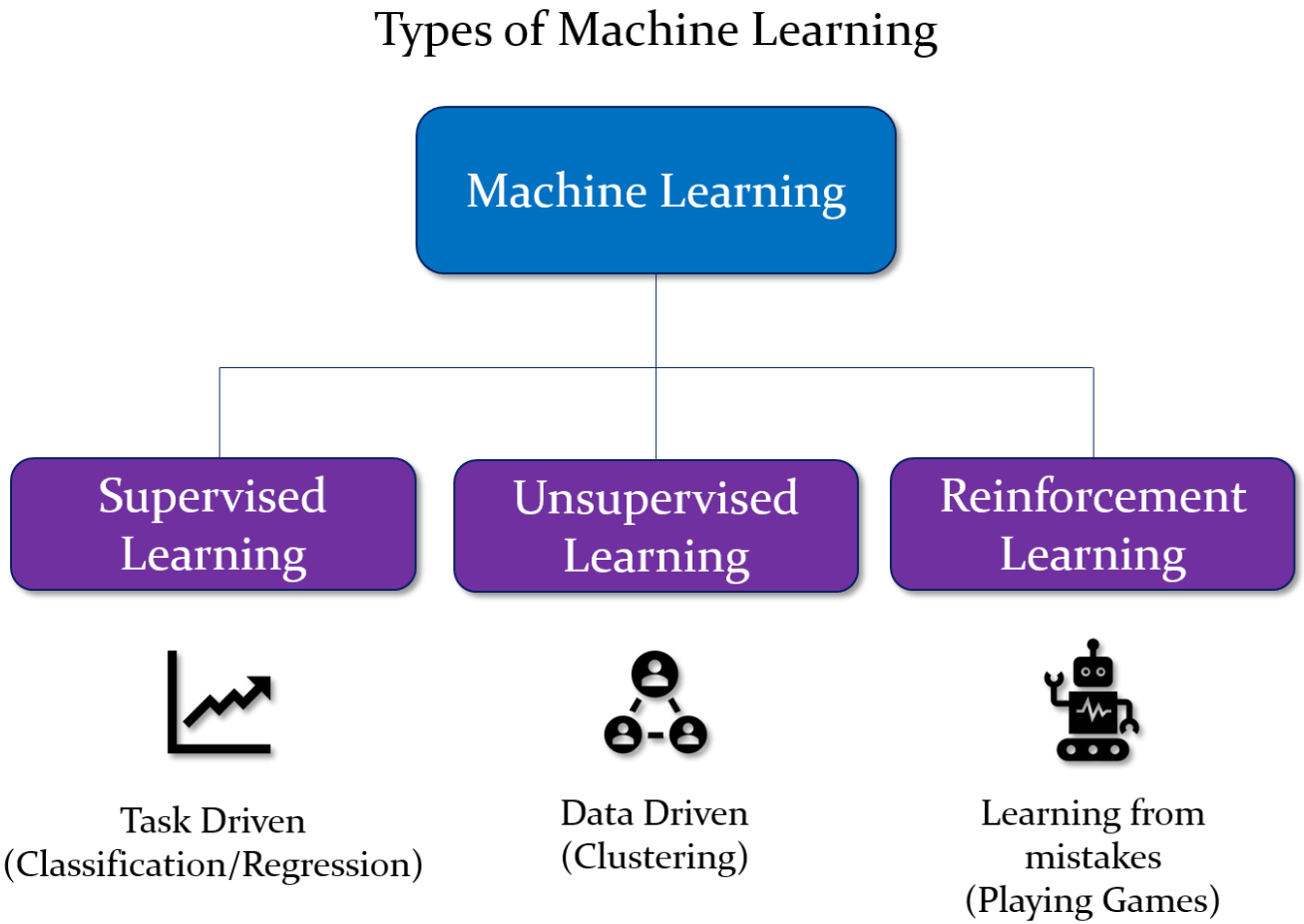
How does Machine Learning work?

Machine learning algorithms are molded on a training dataset to create a model.

As new input data is introduced to the trained ML algorithm, it uses the developed model to make a prediction.



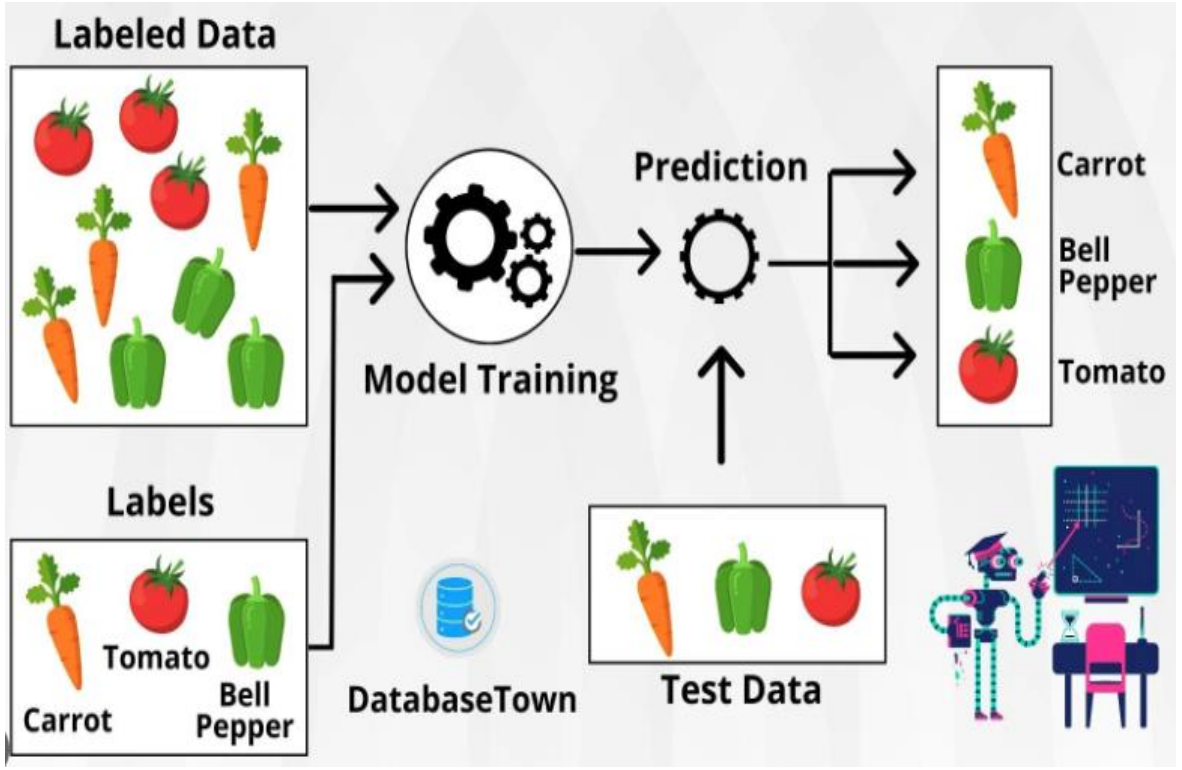
Types of Machine Learning



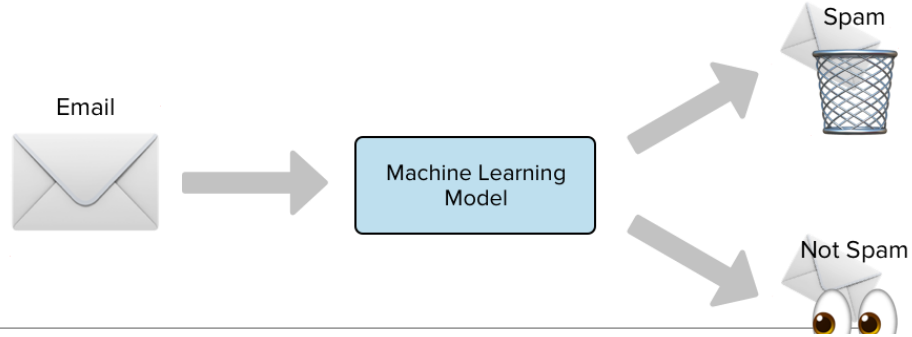
Types of Machine Learning

1. Supervised Machine Learning:

- This type of ML involves supervision, where machines are trained on **labeled datasets** and enabled to predict outputs based on the provided training.
- The labeled dataset specifies that some input and output parameters are already **mapped**.
- Hence, the machine is trained with the input and corresponding output.
- A device is made to **predict** the outcome using the test dataset in subsequent phases.



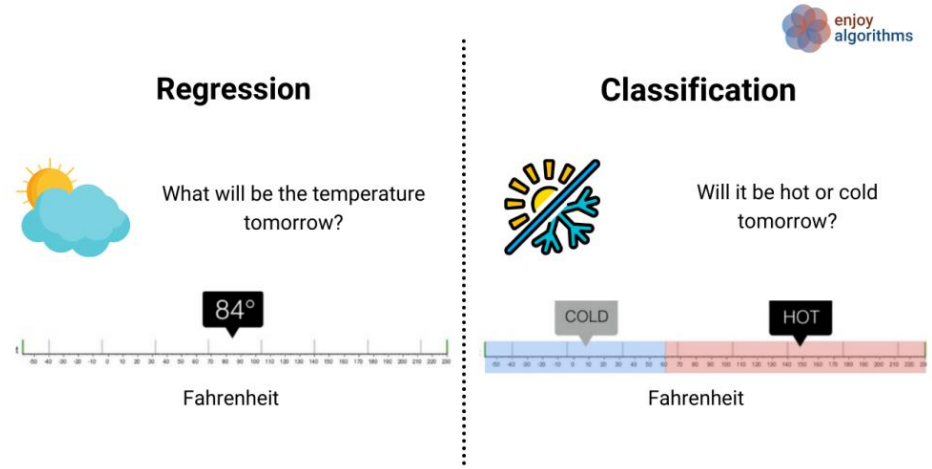
Types of Supervised Machine Learning

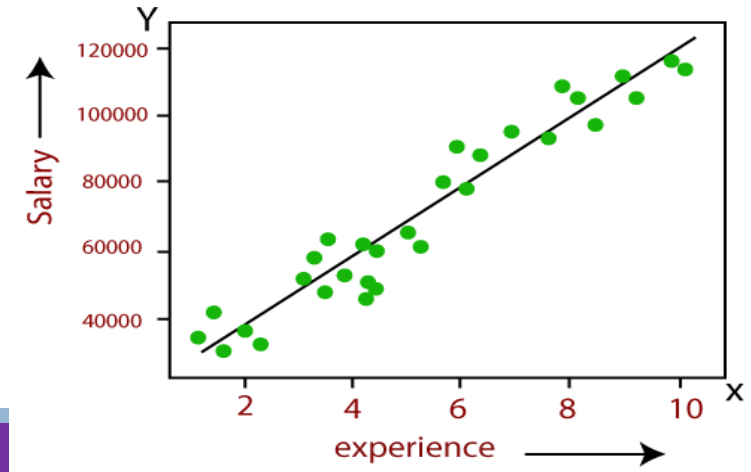


Supervised machine learning is further classified into two broad categories:

Classification: These refer to algorithms that address classification problems where the output variable is **categorical**; for example, yes or no, true or false, male or female, etc. Real-world applications of this category are evident in spam detection and email filtering.

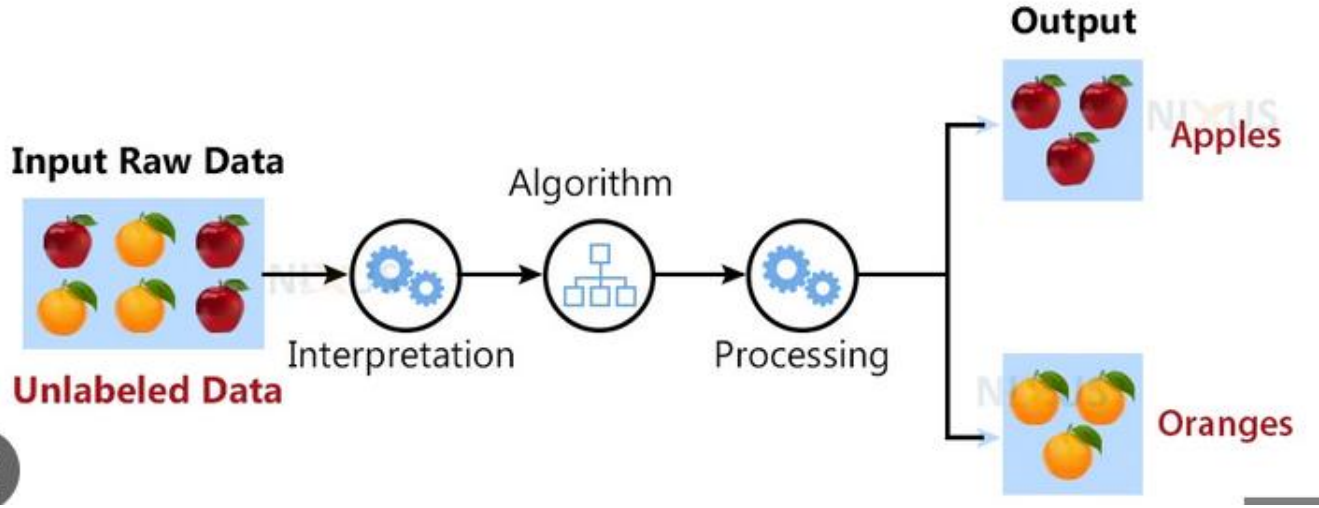
Regression: Regression algorithms handle regression problems where **input and output variables have a linear relationship**. These are known to predict continuous output variables. Examples include weather prediction, market trend analysis, etc.

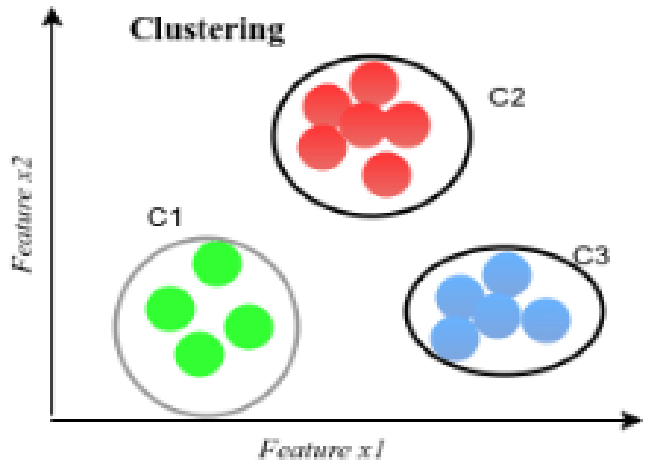




2. Unsupervised machine learning

- Machine is trained using an unlabeled dataset and is enabled to predict the output without any supervision.
- Provide the machine with data and ask to look for **hidden features and cluster the data in a way that makes sense.**
- An unsupervised learning algorithm aims to group the unsorted dataset based on the input's similarities, differences, and patterns.



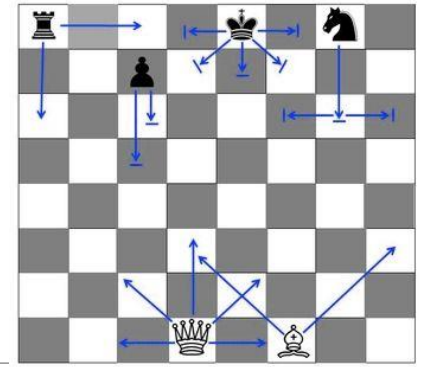


RAMAIAH
Institute of Technology

Parameters	Supervised machine learning	Unsupervised machine learning
Input Data	Algorithms are trained using labeled data.	Algorithms are used against data that is not labeled
Computational Complexity	Simpler method	Computationally complex
Accuracy	Highly accurate	Less accurate
No. of classes	No. of classes is known	No. of classes is not known
Data Analysis	Uses offline analysis	Uses real-time analysis of data
Algorithms used	Linear and Logistics regression, Random forest, Support Vector Machine, Neural Network, etc.	K-Means clustering, Hierarchical clustering, Apriori algorithm, etc.

RAMAIAH
Institute of Technology

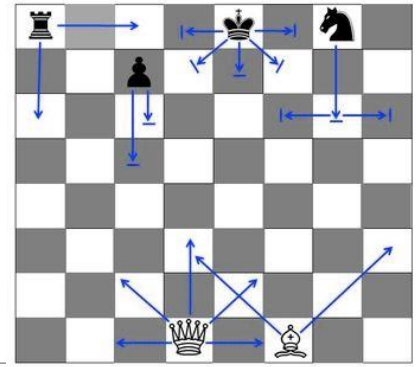
Learning from Rewards



- Consider the problem of learning to play chess. Let's imagine treating this as a **supervised** learning problem.
- The chess-playing agent function takes **as input a board position** and **returns a move**, so we train this function by supplying examples of chess positions, each labeled with the correct move. Available databases of several **million grandmaster games**, each a sequence of **positions and moves**.
- The moves made by the winner are, with few exceptions. The problem is that there are relatively few examples (about 10^{8}) compared to the space of all possible chess positions (about 10^{40}).
- In a new game, one soon encounters positions that are sig**nificantly different from those in the database,** and the trained agent function is likely to fail

RAMAIAH
Institute of Technology

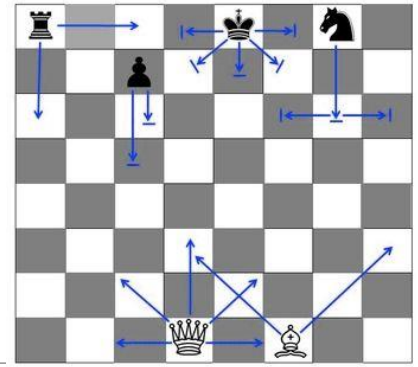
Learning from Rewards



- An alternative is reinforcement learning (RL), in which an **agent interacts with the world and periodically receives rewards (or, in the terminology of psychology, reinforcements) that reflect how well it is doing**.
- For example, in chess the reward is 1 for winning, 0 for losing, and ½ for a draw.

RAMAIAH
Institute of Technology

Learning from Rewards



- Categorize of the approaches:

1. **Model-Based Reinforcement Learning**
2. **Model-Free RL**

RAMAIAH Institute of Technology

Learning from Rewards

1. **Model-Based Reinforcement Learning**

- **Model Learning**: The agent learns a model of the environment that predicts the next state and reward given the current state and action.
- **Planning**: The agent uses the learned model to simulate and evaluate potential future actions to choose the best policy.

This model predicts:

State Transitions: Given a state **s** and action **a**, the model predicts the next state **s'**.

Reward Function: It predicts the reward **r** for a given state-action pair.

RAMAIAH
Institute of Technology

Model Based Learning example

Scenario: Autonomous Navigation in a Complex Environment

Imagine a scenario where an **autonomous drone** is tasked with navigating through a complex and dynamic environment, such as a forest, to deliver **medical supplies** to a remote location. The environment is filled with **obstacles** like trees, branches, and varying terrain, making it crucial for the drone to plan its path efficiently and adapt quickly to any changes.

Complex Environment Modeling: Dynamic Obstacles, Terrain Changes.
Efficient Planning and Adaptation: Simulated Experiences, Real-Time Adjustments.
Safety and Resource Optimization: Collision Avoidance, Battery Efficiency.

RAMAIAH
Institute of Technology

Learning from Rewards

Model-Free Reinforcement Learning

No explicit model: The agent does not construct or use a model of the environment.

Direct learning: The agent learns policies or value functions directly from experiences.

This comes in one of two varieties:

1. ACTION-UTILITY LEARNING
2. POLICY SEARCH

Learning from Rewards

ACTION-UTILITY LEARNING:

- The most common form of action-utility learning is Q-learning, where the agent learns a Q-function **Q(s,a)**, or quality-function, denoting the sum of rewards from state **s** onward if action **a** is taken.
- Given a Q-function, the agent can choose what to do in **s** by finding the action **a** with the highest Q-value.

POLICY SEARCH:

- The agent learns a policy that maps directly from states to actions.
- Same as reflex agent.

RAMAIAH
Institute of Technology

Learning from Rewards

Scenario: Learning to Play a Novel Video Game

Consider a scenario where an artificial intelligence (AI) agent is **learning to play a new**, highly complex video game that has just been released. The game involves a vast, open-world environment with numerous interactive elements, characters, and intricate gameplay mechanics. The game world is detailed and unpredictable, with events and interactions that cannot be easily modeled.

Why Model-Free RL is Suitable?

- **Highly Complex and Unpredictable Environment**: Unmodelable Dynamics, Rich, Diverse Experiences
- **Direct Learning from Interactions**, Trial-and-Error, Adaptation to Game Mechanics
- **Exploration of Unknown Strategies**: Discovering Optimal Policies,Learning from Rewards:

RAMAIAH
Institute of Technology

Learning from Rewards

Aspect	Model-Based RL	Model-Free RL
Use of Environment	Builds a model of the environment.	No explicit model of the environment.
Learning Speed	Faster learning (sample-efficient).	Slower learning (sample-inefficient).
Complexity	More complex (requires modeling).	Simpler (direct learning).
Exploration	Plans exploration using the model.	Trial-and-error exploration.
Examples	AlphaGo, Dyna-Q, Planning-based RL.	Q-Learning, DQN, Policy Gradients.

RAMAIAH Institute of Technology

Passive Reinforcement Learning

- Simple case of a fully observable environment with a small number of actions and states, in which an agent already has a **fixed policy π (s)** that determines its actions.
- The agent is trying to learn the utility function **U^{π}(s),**the expected total discounted reward if policy is executed beginning in state **s**. This is called as a **passive learning agent**.
- **Transition model P(s'|s,a)-**Specifies the probability of reaching state from state after doing action ;
- **Reward function R(s,a,s')-** Specifies the reward for each transition.
- The passive learning agent does not know Transition Model and Reward Function.

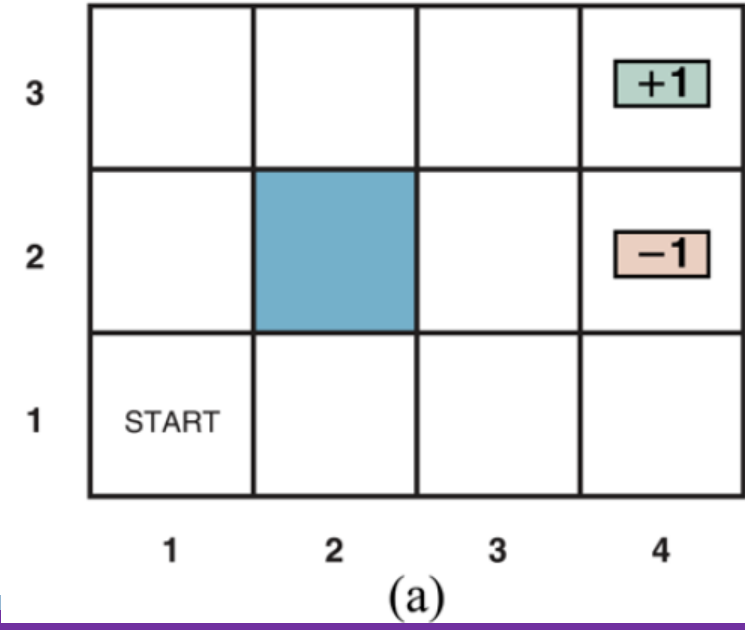
Passive Reinforcement Learning

Suppose that an agent is situated in the 4×3 environment shown in Figure 17.1(a). Beginning in the start state, it must choose an action at each time step. The interaction with the environment terminates when the agent reaches one of the goal states, marked $+1$ or -1. Just as for search problems, the actions available to the agent in each state are given by ACTIONS(s), sometimes abbreviated to $A(s)$; in the 4×3 environment, the actions in every state are *Up*, *Down*, *Left*, and *Right*. We assume for now that the environment is **fully observable**, so that the agent always knows where it is.

Fixed Policy is "Move Right if Possible, otherwise move Up

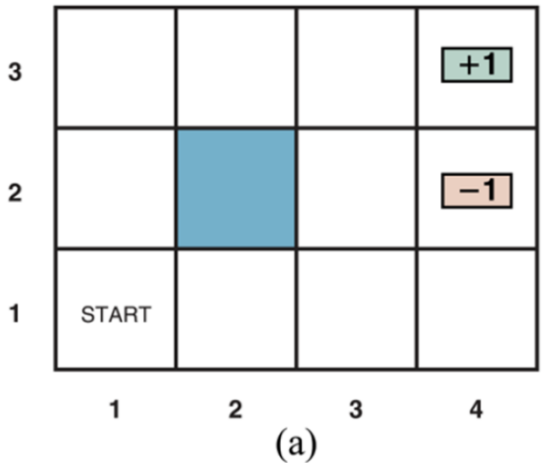
If the environment were deterministic, a solution would be easy: [*Up*, *Up*, *Right*, *Right*, *Right*].

Unfortunately, the environment won't always go along with this solution, because the actions are unreliable.



Passive Reinforcement Learning

- Example : The world shows the optimal policies for world and the corresponding utilities.
- The agent executes a set of trials in the environment using its policy .
- In each trial, the agent starts in state (1,1) and experiences a sequence of state transitions until it reaches one of the terminal states, (4,2) or (4,3).
- Its percepts supply both the current state and the reward received for the transition that just occurred to reach that state. Typical trials might look like this:



(a)

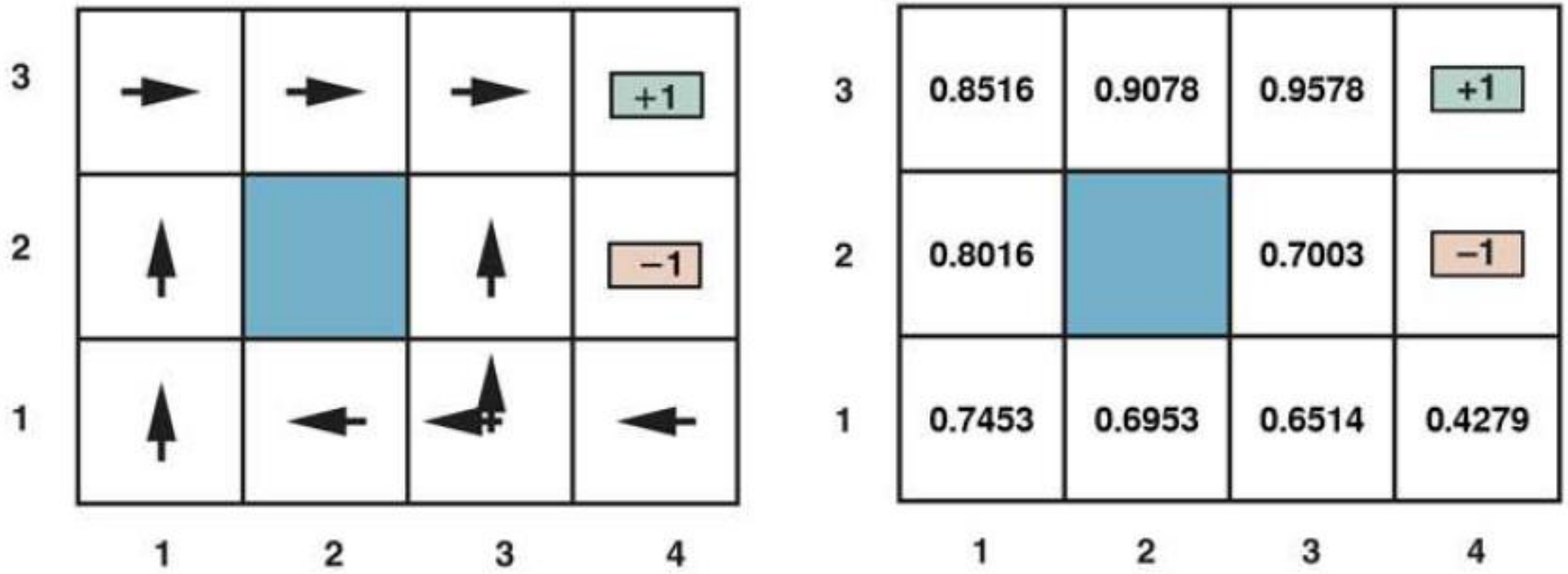
$$(1,1) \xrightarrow[Up]{-.04} (1,2) \xrightarrow[Up]{-.04} (1,3) \xrightarrow[Right]{-.04} (1,2) \xrightarrow[Up]{-.04} (1,3) \xrightarrow[Right]{-.04} (2,3) \xrightarrow[Right]{-.04} (3,3) \xrightarrow[Right]{+1} (4,3)$$

$$(1,1) \xrightarrow[Up]{-.04} (1,2) \xrightarrow[Up]{-.04} (1,3) \xrightarrow[Right]{-.04} (2,3) \xrightarrow[Right]{-.04} (3,3) \xrightarrow[Right]{-.04} (3,2) \xrightarrow[Up]{-.04} (3,3) \xrightarrow[Right]{+1} (4,3)$$

$$(1,1) \xrightarrow[Up]{-.04} (1,2) \xrightarrow[Up]{-.04} (1,3) \xrightarrow[Right]{-.04} (2,3) \xrightarrow[Right]{-.04} (3,3) \xrightarrow[Right]{-.04} (3,2) \xrightarrow[Up]{-1} (4,2)$$

RAMAIAH
Institute of Technology

Passive Reinforcement Learning



(a) The optimal policies for the stochastic environment with $R(s,a,s') = -0.04$ for transitions between nonterminal states. There are two policies because in state (3,1) both $Left$ and Up are optimal. We saw this before in Figure 17.2. (b) The utilities of the states in the 4×3 world, given policy π.

RAMAIAH
Institute of Technology

Passive Reinforcement Learning

The objective is to use the information about rewards to learn the expected utility associated with each nonterminal state .
The utility $\mathbf{U^{\pi}(s)}$ is defined to be the expected sum of (discounted) rewards obtained if policy is followed.

$$U^{\pi}(s) = E\left[\sum_{t=0}^{\infty} \gamma^t R(S_t, \pi(S_t), S_{t+1})\right]$$

where:

- S_t: State at time t.
- $\pi(S_t)$: Action chosen by the policy π in state S_t.
- $R(S_t, \pi(S_t), S_{t+1})$: Reward for transitioning from S_t to S_{t+1}.
- γ: Discount factor ($0 \leq \gamma \leq 1$) that weights future rewards less than immediate rewards.

RAMAIAH Institute of Technology

Passive Reinforcement Learning

```
+---+---+---+
| S1| S2| S3|
|   |   | T | (+10 at S7)
+---+---+---+
| S4| S5| S6|
|   |   |   |
+---+---+---+
| S7| S8| S9|
|+10|   |-10|
+---+---+---+
```

Example Scenario

An agent is navigating a **gridworld environment** to evaluate a fixed policy. The gridworld is as follows:

Gridworld Setup:

- A 3×3 grid with states labeled $S_1, S_2, \ldots, S_9$.
- Start state: S_1.
- Terminal states: S_7 (reward = +10) and S_9 (reward = -10).
- All other states have a reward of 0.
- The fixed policy is **"Move Right if possible; otherwise move Down."**

RAMAIAH Institute of Technology

```
+---+---+---+
| S1| S2| S3|
|   |   | T | (+10 at S7)
+---+---+---+
| S4| S5| S6|
|   |   |   |
+---+---+---+
| S7| S8| S9|
|+10|   |-10|
+---+---+---+
```

Solution

Step 1: Initialize Values

- Start with an arbitrary estimate for $V(S)$, e.g., $V(S) = 0$ for all states.

Step 2: Simulate the Policy

- Use the policy to generate episodes (sequences of states, actions, and rewards) by interacting with the environment.

Example Episode (Simulated):

1. Start at S_1:
    - Fixed policy: Move Right → Transition to S_2, Reward = 0.
2. At S_2:
    - Fixed policy: Move Right → Transition to S_3, Reward = 0.
3. At S_3:
    - Fixed policy: Move Down → Transition to S_6, Reward = 0.

Passive Reinforcement Learning

```
+---+---+---+
| S1| S2| S3|
|   |   | T | (+10 at S7)
+---+---+---+
| S4| S5| S6|
|   |   |   |
+---+---+---+
| S7| S8| S9|
|+10|   |-10|
+---+---+---+
```

4. At S_6:

- Fixed policy: Move Down → Transition to S_9, Reward = -10 (terminal).

Episode: $[S_1, S_2, S_3, S_6, S_9]$, Total Reward = -10.

RAMAIAH
Institute of Technology

Direct Utility Estimation

The idea of direct utility estimation is that the utility of a state is defined as the expected total reward from that state onward (called the expected reward-to -go), and that each trial provides a sample of this quantity for each state visited.

The utility of a state is determined by the reward and the expected utility of the successor states.

RAMAIAH Institute of Technology

Direct Utility Estimation

1. **Utility of a State:** The utility ($U_\pi(s)$) of a state under a policy π is the expected total **discounted reward** obtained from that state onward:

$$U_\pi(s) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R(S_t) \mid S_0 = s, \pi\right],$$

where:

- $R(S_t)$: Reward received at time t,
- γ: Discount factor ($0 \leq \gamma \leq 1$),
- $S_0 = s$: Initial state,
- π: Fixed policy.

2. **Reward-to-Go (G_t):** For any state s, the **reward-to-go** is the total discounted reward starting from that state:

$$G_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \dots$$

RAMAIAH Institute of Technology

Direct

Example of Direct Utility Estimation

Scenario: Gridworld

A robot navigates a 3×3 grid following a fixed policy:

Grid Layout (Rewards in Parentheses):

diff

```
+---------+---------+---------+
| (1,1): -1 | (1,2): -1 | (1,3): +10 (T) |
+---------+---------+---------+
| (2,1): -1 | (2,2): -1 | (2,3): -1 |
+---------+---------+---------+
| (3,1): -1 | (3,2): -1 | (3,3): -10 (T)|
+---------+---------+---------+
```

- **Rewards:**
  - Terminal states $(1, 3)$ and $(3, 3)$ have rewards $+10$ and -10, respectively
  - All other states have a reward of -1.
- **Discount Factor (γ):**
  - $\gamma = 0.9$.
- **Policy (π):**
  - Always move **Right**, then **Down** if Right is unavailable.

Step 1: Simulate an Episode

Run an episode starting from $(1,1)$:

- Path: $(1,1) \rightarrow (1,2) \rightarrow (1,3)$ (terminal).
- Rewards: $-1, -1, +10$.

Step 2: Calculate Reward-to-Go

For each visited state, calculate G_t:

1. At $(1,1)$:

$$G_0 = -1 + 0.9(-1) + 0.9^2(10) = -1 - 0.9 + 8.$$

2. At $(1,2)$:

$$G_1 = -1 + 0.9(10) = -1 + 9 = 8.$$

3. At $(1,3)$:

$$G_2 = +10.$$

Grid Layout (Rewards in Parentheses):

```
diff

+--------+--------+--------+
| (1,1): -1 | (1,2): -1  (1,3): +10 (T) |
+--------+--------+--------+
| (2,1): -1 | (2,2): -1 | (2,3): -1   |
+--------+--------+--------+
| (3,1): -1  (3,2): -1   | (3,3): -10 (T)|
+--------+--------+--------+
```

RAMAIAH
Institute of Technology

Adaptive Dynamic Programming

What is Dynamic Programming

Dynamic Programming (DP) is a method for solving complex problems by breaking them down into smaller, overlapping subproblems and solving each subproblem only once. The solutions to these subproblems are stored (memoized) and reused to solve larger problems efficiently.

RAMAIAH
Institute of Technology

Adaptive Dynamic Programming

ADP is a more advanced method that leverages the principles of **Dynamic Programming (DP)** while incorporating **adaptation** to learn and optimize policies in changing or partially unknown environments.

Key Characteristics:

- ADP often uses or **learns** a model of the environment, including state transition probabilities and rewards.
- The model is either **known beforehand** (in model-based approaches) or approximated during learning.

Adaptive Dynamic Programming

Popular Methods and Techniques

Dynamic Programming (Classical):

Bellman Equation, Policy Iteration, Value Iteration.

Neuro-Dynamic Programming:

Incorporates neural networks for function approximation.

Examples: Deep Q-Networks (DQN), Deep Deterministic Policy Gradient (DDPG).

RAMAIAH Institute of Technology

Adaptive Dynamic Programming

Value Function and Policy Update:

ADP relies on the **Bellman equation** to iteratively update the value function:

$$V(s) = \max_a \left[R(s,a) + \gamma \sum_{s'} P(s'|s,a) V(s') \right]$$

Policies are improved using **policy iteration** or **value iteration**.

Adaptation: ADP adapts to environmental changes dynamically by updating the transition and reward models or adjusting value estimates.

RAMAIAH Institute of Technology

Adaptive Dynamic Programming

Value Function and Policy Update:

ADP relies on the Bellman equation to iteratively update the value function:

$$V(s) = \max_a \left[R(s,a) + \gamma \sum_{s'} P(s'|s,a) V(s') \right]$$

- $V(s)$: Value of state s — the total expected reward when starting in state s and acting optimally.
- $R(s,a)$: Immediate reward received after taking action a in state s.
- γ: Discount factor ($0 \leq \gamma \leq 1$), which determines the importance of future rewards.
- $P(s'|s,a)$: Probability of transitioning to state s' when taking action a in state s.
- $\max_a$: Chooses the action a that maximizes the value.

Real-Time Example: Grid World

Imagine a **robot** navigating a **3x3 grid world** with the following features:

S		
		G +10

1. **States**:
   - Each grid cell is a state s.
   - Start state: $(0, 0)$ (top-left corner).
   - Goal state: $(2, 2)$ (bottom-right corner).
2. **Actions**:
   - **Up, Down, Left, Right**.
3. **Rewards**:
   - Reaching the goal state gives $R = +10$.
   - Each move has a small penalty $R = -1$ to encourage the robot to reach the goal quickly.
4. **Transition Probability**:
   - The robot succeeds in moving in the intended direction with a probability of 0.8.
   - It moves to a random adjacent cell (unintended move) with probability 0.2.
5. **Discount Factor**:
   - $\gamma = 0.9$ (future rewards are less important than immediate rewards).

RAMAIAH Institute of Technolc

Ad

Steps to Solve Using Bellman Equation

1. **Initialize the Value Function**:
   - Start with $V(s) = 0$ for all states.
2. **Iteratively Update the Value Function**:
   - For each state s, calculate $V(s)$ using the Bellman equation:

$$V(s) = \max_a \left[R(s,a) + \gamma \sum_{s'} P(s'|s,a) V(s') \right].$$

S		
		G +10

   - For example, in state $(0,0)$, suppose the robot considers moving **Right**:
     - Reward: $R(s,a) = -1$,
     - Transition probabilities:
       - $P((0,1)|(0,0), \text{Right}) = 0.8$,
       - $P((1,0)|(0,0), \text{Right}) = 0.2$.

     Plugging into the equation:

$$V((0,0)) = \max_{\text{Right}} \left[-1 + 0.9 \left(0.8 V((0,1)) + 0.2 V((1,0)) \right) \right].$$

3. **Repeat Until Convergence**:
   - Continue updating the value function $V(s)$ for all states until the values stabilize (the change in $V(s)$ is below a small threshold).

RAMAIAH Institute of Technology

Temporal Difference Learning

- TD Learning is a class of RL algorithms used to estimate **utility/ value functions**.
- It **updates value estimates** incrementally based on the difference (or error) between consecutive value predictions, known as the **TD error**.

The key features of TD Learning are:

- It **learns from incomplete episodes** (bootstrapping).
- It is **model-free**, meaning no prior knowledge of the environment dynamics is required.
- It **updates estimates online** after every action or time step.

TD Update Rule

The TD Learning update rule for state values $V(s)$ is:

$$V(s) \leftarrow V(s) + \alpha \left[r + \gamma V(s') - V(s) \right].$$

Parameters:

- $V(s)$: The current estimated value of state s.
- r: The immediate reward after transitioning from state s to s'.
- $V(s')$: The estimated value of the next state s'.
- γ: The **discount factor** ($0 \leq \gamma \leq 1$) that balances immediate and future rewards.
- α: The **learning rate** ($0 < \alpha \leq 1$) that determines how quickly the value function is updated.

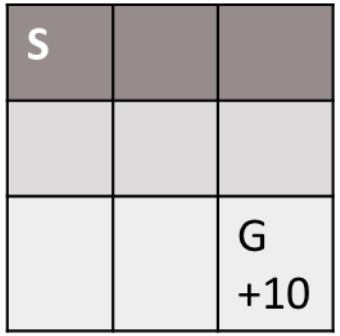
The term $\left[r + \gamma V(s') - V(s) \right]$ is called the **TD error**.

Temporal Difference Learning

Real-Time Example: Grid World

Imagine a robot navigating a **3x3 grid world** to reach a goal. The environment is:

- **States**: Grid positions $(0, 0)$ to $(2, 2)$.
- **Actions**: Up, Down, Left, Right.
- **Rewards**:
  - Reaching the goal gives $+10$,
  - Each move costs -1.
- **Discount Factor**: $\gamma = 0.9$,
- **Learning Rate**: $\alpha = 0.5$.



RAMAIAH Institute of Technology

TD(0) Example

1. **Start State:** $(0, 0)$, Action: **Right**.
   - Immediate reward $r = -1$,
   - Next state $(0, 1)$.
2. Update State Value $V((0, 0))$ using the TD(0) rule:

$$V((0,0)) \leftarrow V((0,0)) + \alpha \left[r + \gamma V((0,1)) - V((0,0))\right]$$

S		
		G +10

Assume:

- $V((0, 0)) = 0$ (initial value),
- $V((0, 1)) = 5$ (estimated value of next state).

Substitute values:

$$V((0,0)) \leftarrow 0 + 0.5\left[-1 + 0.9 \cdot 5 - 0\right].$$

Simplify:

$$V((0,0)) = 0 + 0.5\left[-1 + 4.5\right].$$

$$V((0,0)) = 0 + 0.5 \cdot 3.5 = 1.75.$$

The updated value of $V((0, 0))$ is **1.75**.

RAMAIAH
Institute of Technology

Active Reinforcement Learning

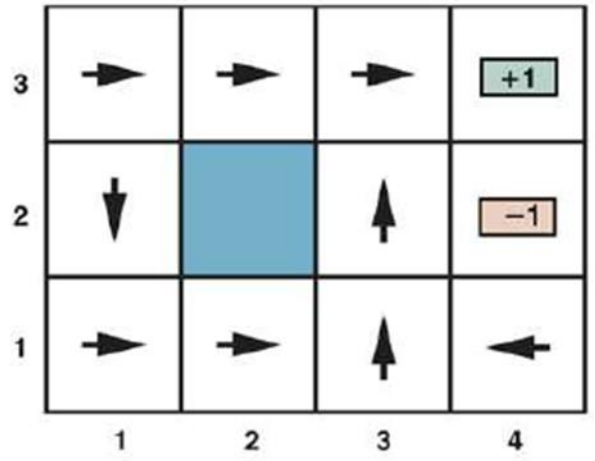
- Active RL is a **variant** of reinforcement learning that incorporates elements of active learning.
- A **passive learning** agent has a **fixed policy** that determines its behavior.
- The **agent actively chooses** the actions based on the current state of the environment. This means that the agent has **complete control over its actions and is free to explore** different options to determine the best way to maximize its reward.

Active Reinforcement Learning

- The learning mechanism used by PASSIVE-ADP-AGENT takes into account the fact that the agent has a choice of actions. The utilities it needs to learn are those defined by the optimal policy; they obey the Bellman equations

$$V(s) = \max_a \left[R(s,a) + \gamma \sum_{s'} P(s'|s,a) V(s') \right]$$

- In third trial, it finds a policy that reaches the reward along the lower route via (2,1), (3,1), (3,2), and (3,3). But never finds the optimal route via (1,2), (1,3), and (2,3).
- This agent is called a **greedy agent**, because it greedily takes the action that it currently believes to be optimal at each step.



RAMAIAH
Institute of Technology

Active Reinforcement Learning

- The greedy agent has **overlooked the fact that actions** do more than provide ***rewards***; they also provide ***information*** in the form of percepts in the resulting states.
- In the real world, one constantly has to decide between continuing in a comfortable existence, versus striking out into the unknown in the hopes of a better life.

RAMAIAH
Institute of Technology

Active Reinforcement Learning

- What are Exploration and Exploitation in Reinforcement Learning

- In reinforcement learning, whenever agents get a situation in which they have to make a difficult choice between whether to **continue the same work or explore something new at a specific time**, then, this situation results in Exploration-Exploitation Dilemma because the knowledge of an agent about the state, actions, rewards and resulting states is always partial.

RAMAIAH
Institute of Technology

Active Reinforcement Learning

- **Exploitation** in Reinforcement Learning

- **Exploitation** is defined as a **greedy approach** in which agents try to get more rewards by using **estimated value but not the actual value**. So, in this technique, agents make the best decision based on current information.

RAMAIAH Institute of Technology

Active Reinforcement Learning

- **Exploration** in Reinforcement Learning
- In exploration techniques, agents primarily focus on **improving their knowledge about each action** instead of getting more rewards so that they can get long-term benefits. So, in this technique, agents work on gathering more information to make the best overall decision.

RAMAIAH
Institute of Technology

Active Reinforcement Learning

- Example 1: Let's say we have a scenario of **online restaurant selection** for food orders, where you have two options to select the restaurant.
- In the **first option**, you can choose your favorite restaurant from where you **ordered food in the past**; this is called **exploitation** because here, you only know information about a specific restaurant.
- **Second** option, you can try a **new restaurant to explore** new varieties and tastes of food, and it is called **exploration**. However, food quality might be better in the first option, but it is also possible that it is more delicious in another restaurant.

RAMAIAH Institute of Technology

Active Reinforcement Learning

- Example 2: Suppose there is a game-playing platform where you can play chess with robots.
- To win this game, you have two choices either **play the move that you believe is best**, and for the other choice, you can **play an experimental move**. Here, the first choice is called **exploitation**, where you know about your game strategy, and the second choice is called **exploration**, where you are exploring your knowledge and playing a new move to win the game.

RAMAIAH Institute of Technology

Active Reinforcement Learning

- **Epsilon Greedy Policy**
- Epsilon greedy policy is defined as a technique to maintain a **balance between exploitation and exploration**. However, to choose between exploration and exploitation, a very simple method is to **select randomly**. This can be done by choosing exploitation most of the time with a little exploration.

RAMAIAH Institute of Technology

Active Reinforcement Learning

- This equation appears to represent a mechanism in reinforcement learning (RL) for balancing **exploration** and **exploitation** while computing an optimal value function.

$$U^+(s) \leftarrow \max_a f\left(\sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma U^+(s')], N(s,a)\right).$$

RAMAIAH
Institute of Technology

Active Reinforcement Learning

$$U^+(s) \leftarrow \max_a f\left(\sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma U^+(s')], N(s,a)\right).$$

1. $U^+(s)$: Represents an updated utility (value) of a state s, considering the best action a.
2. $\max_a$: The process of selecting the action a that maximizes the value function for a given state.
3. $P(s'|s,a)$: The transition probability of reaching state s' from state s after taking action a.
4. $R(s,a,s')$: The reward received after transitioning from state s to state s' by taking action a.
5. γ: A discount factor (between 0 and 1) that weighs the importance of future rewards compared to immediate rewards.
6. $U^+(s')$: The current estimated utility of the next state s'.
7. $N(s,a)$: Represents the visitation count for state-action pair (s,a), which influences exploration.

Scenario: Robot Vacuum Cleaner

The robot's goal is to clean the house efficiently by visiting different rooms. However, it must also learn over time which areas accumulate more dirt and optimize its path accordingly.

States (s):

Each room or location in the house (e.g., Living Room, Kitchen, Bedroom).

Actions (a):

The robot can move in four directions: "up," "down," "left," or "right."

Rewards ($R(s, a, s')$):

- A reward is given for cleaning a dirty spot.
- A penalty is given for hitting walls or obstacles.
- A small positive reward might also be given for efficiently reaching unexplored areas.

Transition Probabilities ($P(s'|s,a)$):

The robot moves deterministically most of the time, but it might occasionally slip or encounter obstacles, which make transitions probabilistic.

How Exploration vs. Exploitation Applies

1. **Exploitation**:
   - Once the robot has mapped the house and knows the high-dirt areas (e.g., the Kitchen accumulates more dirt), it focuses on revisiting those areas because they maximize its rewards.
   - This is computed by maximizing:

$$\sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma U^{+}(s')]$$

     Example: The robot learns that cleaning under the dining table in the Kitchen consistently yields high rewards because food crumbs often fall there.

RAMAIAH
Institute of Technology

Active Reinforcement Learning

2. **Exploration**:

- Initially, the robot doesn't know the layout of the house or where the most dirt accumulates. So, it must explore new areas by taking actions it hasn't tried before.
- Actions with lower visitation counts ($N(s, a)$) are prioritized by the exploration bonus in f. This encourages the robot to explore unknown rooms or paths. Example: The robot tries moving into the hallway, even if it's uncertain about the rewards, to check if dirt is accumulating there.

RAMAIAH
Institute of Technology

Real-Time Example in Action:

- **Initial Stage (Exploration):**
  - The robot begins in the Living Room. It explores all possible directions to learn the layout of the house.
  - If it has rarely visited the Bedroom, the function f assigns a higher value to actions leading there, despite not knowing if it will find dirt. The exploration ensures that all parts of the house are visited.
- **Later Stage (Exploitation):**
  - After several runs, the robot learns that the Kitchen and Living Room corners collect the most dirt. It focuses on visiting those areas frequently, maximizing rewards for cleaning dirt efficiently.
  - Exploration decreases because $N(s, a)$ for those actions is now high.

Safe Exploration

Safe exploration

So far we have assumed that an agent is free to explore as it wishes—that any negative rewards serve only to improve its model of the world.

Example: If we play a game of **chess** and lose, we suffer no damage (except perhaps to our pride), and whatever we learned will make us a better player in the next game.

In a simulation environment for a **self driving car**, we could explore the limits of the car's performance, and any accidents give us more information. If the car crashes, we just hit the reset button

RAMAIAH
Institute of Technology

Safe Exploration

Unfortunately, the real world is less forgiving.

Many actions are **irreversible**, no subsequent sequence of actions can restore the state to what it was before the irreversible action was taken.

In the worst case, the agent enters an **absorbing** state where no actions have any effect and no rewards are received.

RAMAIAH
Institute of Technology

Safe Exploration

In many **practical settings**, we cannot afford to have our agents taking irreversible actions or entering absorbing states.

For example, an agent learning to **drive in a real car** should **avoid** taking actions that might lead to any of the following:

- states with large negative rewards, such as serious car crashes;
- states from which there is no escape, such as driving the car into a **deep ditch**;
- states that **permanently** limit future rewards, such as damaging the car's engine so that its maximum speed is reduced

RAMAIAH
Institute of Technology

Safe Exploration

A better idea would be to choose a policy **that works reasonably well** for the whole range of models that have a reasonable chance of being the **true model**, even if the policy happens to be suboptimal for the maximum-likelihood model. There are mathematical approaches that have this flavor

RAMAIAH Institute of Technology

Safe Exploration

A better idea would be to choose a policy that **works reasonably well for the whole range of models** that have a reasonable chance of being the true model

Bayesian reinforcement learning, Bayesian reinforcement learning (Bayesian RL) is a formalism that uses Bayesian inference to address uncertainty in reinforcement learning (RL) environments.

$$\pi^* = \underset{\pi}{\operatorname{argmax}} \sum_h P(h|\mathbf{e}) U_h^\pi.$$

RAMAIAH
Institute of Technology

Safe Exploration

Bayesian reinforcement learning, assumes a prior probability **P(h)**over hypotheses **h** about what the true model is; the posterior probability **P(h|e)** is obtained in the usual way by Bayes' rule given the observations to date. Then, if the agent has decided to stop learning, the optimal policy is the one that gives the highest expected utility. Let U_h^π be the expected utility, averaged over all possible start states, obtained by executing policy in model .

$$\pi^* = \operatorname*{argmax}_\pi \sum_h P(h|\mathbf{e}) U_h^\pi .$$

RAMAIAH Institute of Technology

Safe Exploration

$$\pi^* = \underset{\pi}{\operatorname{argmax}} \sum_h P(h|\mathbf{e}) U_h^\pi.$$

Components:

1. π^*: The optimal policy that we aim to find.
2. π: A candidate policy.
3. $\sum_h$: A summation over all possible hypotheses h (different models of the environment).
4. $P(h|e)$: The posterior probability of a hypothesis h, given evidence e.
    - This reflects our belief about the environment after observing some evidence.
5. U_h^π: The utility (or expected reward) of a policy π under hypothesis h.
6. $\arg\max$: Chooses the policy π that maximizes the expected utility.

This equation expresses that the optimal policy is the one that maximizes the expected utility over all possible hypotheses, weighted by their posterior probabilities.

RAMAIAH Institute of Technology

Robot Navigation

Scenario: A robot navigates a 5x5 grid world to reach a goal. The terrain type affects how likely it is to successfully move in the intended direction.

Step 1: Define Hypotheses (h)

The robot considers two hypotheses about the environment:

- h_1: The terrain is **smooth** (90% movement success rate).
- h_2: The terrain is **rough** (50% movement success rate).

Step 2: Assign Prior Probabilities

Before observing anything, the robot assumes both models are equally likely:

$$P(h_1) = 0.5,\ P(h_2) = 0.5$$

Step 3: Take Actions and Observe Outcomes (e)

- The robot starts at the bottom-left of the grid.
- It attempts to move up and succeeds five times in a row.
- Observation e: 5 successful moves.

Step 4: Update Beliefs Using Bayes' Rule

Using the likelihood of observing e under each hypothesis:

1. Likelihood of e under h_1:

$$P(e \mid h_1) = (0.9)^5 = 0.59049$$

2. Likelihood of e under h_2:

$$P(e \mid h_2) = (0.5)^5 = 0.03125$$

3. Compute posterior probabilities:

$$P(h_1 \mid e) = \frac{P(e \mid h_1)P(h_1)}{P(e)}$$

$$P(h_2 \mid e) = \frac{P(e \mid h_2)P(h_2)}{P(e)}$$

Since $P(e)$ normalizes the probabilities:

$$P(h_1 \mid e) = \frac{0.59049 \cdot 0.5}{0.59049 \cdot 0.5 + 0.03125 \cdot 0.5} \approx 0.95$$

$$P(h_2 \mid e) = 1 - P(h_1 \mid e) = 0.05$$

Now the robot strongly believes the terrain is smooth (h_1).

RAMAIAH
Institute of Technology

Temporal-Difference Q-Learning

- **Temporal-Difference Q-Learning** is one of the foundational algorithms in reinforcement learning.
- It is a model-free, off-policy learning method that enables an agent to learn optimal action policies by directly estimating the **Q-values** (action-value function) using experience.
- It learns Q-values by updating estimates incrementally, even before the final outcome of an episode is known.

Q-Learning Algorithm

The **Q-value** for a state-action pair (s, a), denoted as $Q(s, a)$, represents the expected total re
when taking action a in state s, and following the optimal policy thereafter.

Update Rule:

The Q-value is updated as follows:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right]$$

Where:

- s_t, a_t: Current state and action.
- r_{t+1}: Immediate reward received after taking action a_t.
- s_{t+1}: Next state.
- α: Learning rate (step size).
- γ: Discount factor (weight of future rewards).
- $\max_a Q(s_{t+1}, a)$: Maximum Q-value of the next state, representing the greedy choice of action.

How It Works

1. **Initialize Q-Table:**
   - Initialize $Q(s, a)$ for all states s and actions a arbitrarily (or with zeros).
2. **Interaction with the Environment:**
   - At each time step:
     - Choose an action a_t in state s_t (using an exploration strategy like ϵ-greedy).
     - Execute a_t and observe r_{t+1} and s_{t+1}.
     - Update $Q(s_t, a_t)$ using the update rule.
3. **Repeat:**
   - Continue interacting with the environment, improving the Q-values iteratively.
4. **Policy Extraction:**
   - Once training is complete, extract the optimal policy by choosing actions that maximize $Q(s, a)$ for each state s:

$$\pi^*(s) = \arg\max_a Q(s, a)$$

Scenario: A Simple 3x3 Gridworld

Environment Description:

- **Grid**: A 3x3 grid (9 states in total).
- **Goal**: Reach the bottom-right cell (state 8) and receive a reward of $+10$.
- **Actions**: Up, Down, Left, Right.
- **Rewards**:
  - -1 for each step.
  - $+10$ for reaching the goal state.
- **Start State**: Top-left corner (state 0).

RAMAIAH
Institute of Technology

Q-Table Representation

State	Up	Down	Left	Right
0	0.0	1.5	0.0	2.3
1	0.0	2.1	1.2	2.7
2	0.0	3.0	2.4	0.0
3	1.3	3.5	0.0	4.2
4	2.0	4.8	3.1	5.6
5	3.2	6.4	4.9	0.0
6	4.8	0.0	0.0	7.2
7	5.9	10.0	6.1	8.3
8	0.0	0.0	0.0	0.0

Explanation of Q-Table Values:

1. **States (Rows):**
   - Each row corresponds to one state (e.g., 0 for top-left, 8 for bottom-right).
2. **Actions (Columns):**
   - Each column represents a possible action:
     - Up, Down, Left, Right.
3. **Q-Values:**
   - Each value represents the expected reward for taking an action in a given state an following the optimal policy.
4. **Terminal State:**
   - For state 8 (goal), all actions have $Q = 0.0$, as no further rewards can be gained.

RAMAIAH
Institute of Technology

How Q-Table Updates Work

The Q-values in the table are updated iteratively during training using the **Q-learning update rule:**

$$Q(s,a) \leftarrow Q(s,a) + \alpha\left[R(s,a) + \gamma \max_{a'} Q(s',a') - Q(s,a)\right]$$

Q-Table Representation

State	Up	Down	Left	Right
0	0.0	1.5	0.0	2.3
1	0.0	2.1	1.2	2.7
2	0.0	3.0	2.4	0.0
3	1.3	3.5	0.0	4.2
4	2.0	4.8	3.1	5.6
5	3.2	6.4	4.9	0.0
6	4.8	0.0	0.0	7.2
7	5.9	10.0	6.1	8.3
8	0.0	0.0	0.0	0.0

- Example:
  - Suppose the agent is in **state 4** and takes the action **Right.**
  - It moves to **state 5**, gets a reward of -1, and the highest Q-value for state 5 is 6.4.
  - With $\alpha = 0.1$ and $\gamma = 0.9$:

$$Q(4,\text{Right}) = Q(4,\text{Right}) + 0.1\left[-1 + 0.9 \cdot 6.4 - Q(4,\text{Right})\right]$$

Advantages of Q-Tables

1. Simple and easy to implement for small state-action spaces.
2. Provides a clear representation of learned Q-values.

Limitations of Q-Tables

1. **Scalability**:
   - For large state-action spaces, Q-Tables become infeasible due to memory constraints.
2. **Continuous State Spaces**:
   - Requires approximations (e.g., function approximators like neural networks in **Deep Q-Learning**).

A Q-Table is a foundational tool in Q-Learning, helping an agent to store and update its knowledge about the environment. While suitable for small-scale problems, advanced methods like Deep Q-Learning are used for complex tasks.

Generalization in Reinforcement Learning (RL)

Generalization in RL refers to the ability of an agent to perform well in unseen states, tasks, or environments, given the knowledge it has acquired during training. In other words, it ensures the learned policy is robust and not overfitted to the training environment.

Key Aspects of Generalization in RL

1. **Overfitting vs. Generalization:**
   - **Overfitting:** The agent learns to perform well in the specific environment or dataset it was trained on but fails in new scenarios.
   - **Generalization:** The agent learns patterns or behaviors that are applicable across a range of tasks or environments.
2. **Challenges:**
   - Large and complex state-action spaces.
   - Limited or biased training data.
   - Variability in environment dynamics.

Techniques to Promote Generalization in RL

Environment Diversity (Domain Randomization):

- Train the agent in multiple variations of the environment, e.g., by changing textures, dynamics, or goals.
- Example: A robot trained on different terrains (smooth, rough) will generalize better to new terrains.

Data Augmentation:

- Apply transformations to observations during training (e.g., rotations, noise, cropping).
- In visual RL, augmenting image inputs can help the agent learn invariant features.

Exploration Strategies:

- Encourage broader exploration during training to expose the agent to diverse states.
- Use methods like curiosity-driven exploration or entropy-based rewards.

Problem: A robot is trained to navigate a room with obstacles to reach a goal.

Without Generalization:

- The robot learns to avoid obstacles based on their specific locations and patterns in the training room.
- When placed in a new room with different obstacles, it fails.

With Generalization:

- The robot learns general principles (e.g., avoid any obstacle, follow gradients toward the goal).
- It successfully navigates new rooms with different obstacle arrangements.

Deep Reinforcement Learning (Deep RL)

Deep Reinforcement Learning (Deep RL) combines the power of **deep learning** and **reinforcement learning (RL)** to solve complex decision-making problems. It uses deep neural networks to approximate policies, value functions, or both, enabling agents to handle large, high-dimensional state and action spaces.

Reinforcement Learning Framework:

- **Agent**: Learns to take actions.
- **Environment**: Provides observations and rewards based on the agent's actions.
- **Policy** (π): The strategy that the agent follows to decide actions.
- **Reward Signal** (R): Feedback from the environment.
- **Value Function** ($V(s)$): Predicts the expected reward for being in a state s.
- **Action-Value Function** ($Q(s, a)$): Predicts the expected reward for taking action a in state s.

Deep Learning:

- Uses deep neural networks to approximate complex functions, such as:
  - $Q(s, a)$: Action-value functions.
  - $\pi(a|s)$: Policies.

Combining Deep Learning with RL:

- Instead of maintaining large tabular representations, neural networks generalize over continuous and high-dimensional state-action spaces.

Key Algorithms in Deep RL

1. Value-Based Methods

- Use neural networks to approximate the value function or Q-function.

Example: Deep Q-Learning (DQN)

2. Policy-Based Methods

- Directly learn the policy $\pi(a|s)$ using neural networks.

Example: Policy Gradient Methods

4. Model-Based Methods

- Learn a model of the environment dynamics $P(s'|s,a)$ and reward $R(s,a)$.
- Use the learned model to plan or simulate rollouts.

Example: Deep Planning Algorithms

- Model Predictive Control (MPC) with neural network-based models.

Challenges in Deep RL

1. **Exploration vs. Exploitation**:
    - Encouraging the agent to explore unseen states while exploiting known high-reward actions.
2. **Sample Efficiency**:
    - Deep RL often requires a large number of interactions with the environment, which can be computationally expensive.
3. **Stability and Convergence**:
    - Training deep networks with RL is less stable than supervised learning due to non-stationary data and correlated updates.

Reward shaping

- Reward shaping in reinforcement learning (RL) refers to the technique of **modifying the reward function to encourage an agent to learn more efficiently or faster.**
- Instead of relying solely on the final reward (or sparse rewards), reward shaping provides **additional intermediate rewards** during the learning process.
- This can guide the agent towards **better behaviors** by giving it more frequent feedback.

Reward shaping

- **Intrinsic vs. Extrinsic Rewards**:
  - **Extrinsic Rewards**: Rewards that come from the **environment**, typically defined by the task at hand (e.g., +1 for reaching the goal, -1 for failure).
  - **Intrinsic Rewards**: Rewards that are **generated by the agent itself to encourage exploration**, curiosity, or learning progress (e.g., rewarding the agent for exploring new states or reducing uncertainty).
- **Shaping the Reward Function**: In reward shaping, the reward function is typically modified by adding a **potential-based term**. This term provides **guidance towards better** actions based on the state and the agent's trajectory.

Reward shaping

- **Potential-based Reward Shaping**: One of the most common ways to apply reward shaping is through potential-based reward shaping (PBRS). The idea is to define a potential function Φ(s) over the states, where the potential is a **value that encourages desirable behaviors**. The shaped reward is then given by:

$$R'(s,a,s') = R(s,a,s') + \gamma \cdot \Phi(s') - \Phi(s)$$

where:
•R(s,a,s′) is the original reward.
•γ is the discount factor.
•Φ(s) is the potential function at state s.

Reward shaping

- The potential function Φ(s) is typically chosen **to reflect how desirable a particular state is (e.g., the proximity to the goal**). Importantly, potential-based shaping **does not change the optimal policy** because the term $\gamma \cdot \Phi(s') - \Phi(s)$ is designed to only modify the rewards in a way that preserves the **relative desirability of actions**.

Example of Reward Shaping: Suppose you have a robotic agent tasked with reaching a goal in a maze. The agent only gets a **reward of +1 when it reaches the goal**. Without reward shaping, the agent may take many steps to find the goal and may only **learn through trial and error**. If you apply reward shaping, you could give the agent small rewards for moving closer to the goal.
For example:+0.1 for each step closer to the goal (based on a distance metric).-0.1 for each step farther from the goal.

RAMAIAH
Institute of Technology

Applications of Reinforcement Learning

Development of reinforcement learning (RL) in the context of **game playing Key milestones**:

Arthur Samuel's Early Work (1952): Samuel pioneered reinforcement learning in the context of checkers, laying the foundation for RL.

Neurogammon (1990):

Developed by Gerry Tesauro for backgammon. It used imitation learning by **analyzing 400 games** Tesauro played against himself. Neurogammon transformed each move (**state-action pairs (s,a,s ′)** into training examples, comparing positions to learn an evaluation function using neural network. Achieved success by **winning the 1989 Computer Olympiad** but could not surpass intermediate-level gameplay.

RAMAIAH
Institute of Technology

Applications of Reinforcement Learning

TD-Gammon (1992): A more advanced system built by Tesauro using TD (temporal difference) learning, influenced by Richard Sutton's research. Used a single-hidden-layer neural network with 80 nodes. After 300,000 training games, TD-Gammon reached **human-expert levels**, comparable to the top three players globally. Kit Woolsey, a top player, praised its positional judgment as superior to his own.

Deep Q-Network (DQN): Developed by DeepMind starting in 2012, this marked the modern era of Deep RL. Unlike traditional board representations, DQN learned directly from raw perceptual inputs. It employed deep neural networks to represent the Q-function, showcasing the advancements in RL techniques.

Applications of Reinforcement Learning

DQN was trained separately on 49 video games, learning to master diverse tasks like:

- Driving simulated race cars.
- Shooting alien spaceships.
- Bouncing balls with paddles.

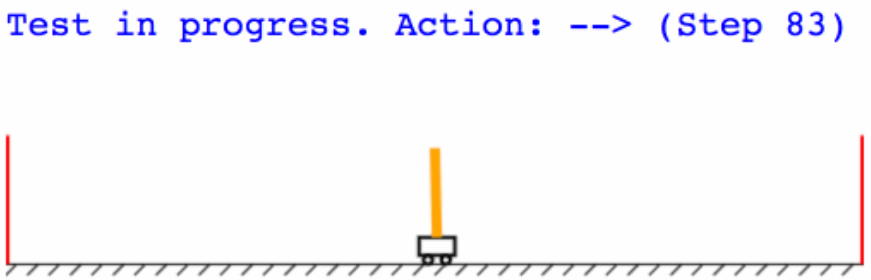
AlphaGo showed how RL can tackle more complex, strategic domains by integrating planning and prediction.

RAMAIAH
Institute of Technology

Applications of Reinforcement Learning in Robot Control

Reinforcement learning (RL) has been extensively applied to **robot control**, enabling robots to perform tasks that require decision-making, adaptability, and interaction with complex environments. Below are key areas and examples of its applications in robot control:

Classic Control ProblemsCart-Pole (Inverted Pendulum): The robot learns to balance a pole on a cart by applying discrete forces (left or right).RL methods like the Boxes algorithm or neural network approximations help solve this problem by learning the optimal actions to keep the pole upright. Helps develop and evaluate algorithms for balancing, decision-making, and controlling dynamic systems,



RAMAIAH
Institute of Technology

Applications of Reinforcement Learning in Robot Control

Boxes Algorithm (1968): Developed by Michie and Chambers using a real cart-pole system. The state space was discretized into "boxes," and reinforcement learning (negative rewards for failures) was used to improve balancing. The system learned to balance the pole for over an hour after 30 trials. **Helicopter Flight Control**: RL has been applied to train autonomous helicopters for stable flight, complex aerobatic maneuvers, and navigation in turbulent conditions.